CSC490 A5

Zhuorui(Jack) Jiang 1008917059
Qixiang(David) Fan 1008847241
Zihan(Jason) Zhao 1010128274
Gabriel Won Bin Lee 1007488504

March 27, 2026

Part One - Profiling Execution Time

We profiled our FastAPI backend using Python's built-in `cProfile` module (`backend/app/profiling.py`). The script authenticates against the running application via `TestClient`, then exercises every major API endpoint — `/health`, `/api/trending`, `/api/market-summary`, `/api/prices`, `/api/stock/{ticker}`, `/api/news`, `/api/earnings`, `/api/sec`, `/api/indicator`, `/api/market-analysis`, `/auth/me`, `/api/watchlist`, and `/api/search` — while the profiler is enabled. Results are sorted by cumulative time and filtered to application code only.

Before: Baseline Profile

The baseline run completed in **31.22 s**. The five slowest application-level functions were:

Function	cumtime (s)	ncalls	Bottleneck
`generate_market_analysis()`	25.979	1	Single Bedrock API call with verbose prompt and 8192 max tokens
`fetch_market_summary()`	4.851	1	20 sequential FRED/yfinance API calls
`get_stock()`	1.392	1	Sequential `fetch_bars()` then `fetch_info()`
`search_tickers()`	0.220	1	No caching; every query hits the yfinance API
`cache.get()`	0.017	10	File I/O on every access; no in-memory layer

Optimizations Applied

1. `fetch_market_summary()` — **Parallelized API calls.** The original implementation called 20 FRED and yfinance endpoints one after another. We refactored the function to dispatch all 20 calls concurrently using `concurrent.futures.ThreadPoolExecutor(max_workers=10)`, then reassemble results in the original category order.

   Why this matters: The market summary page is one of the first things users see when they open the app. A nearly 5-second wait for the dashboard to populate feels sluggish and risks users abandoning the page before the data even loads. Cutting this to 0.6 s makes the dashboard feel near-instant.

2. `generate_market_analysis()` — **Reduced prompt size and token budget.** The Bedrock (Claude) prompt was condensed from a verbose multi-paragraph instruction to a compact schema-driven prompt, cutting input token count. We also lowered `maxTokens` from 8192 to 4096 (the actual responses are well under 4K tokens) and reused a singleton `boto3` client instead of constructing a new one per request.

   Why this matters: This endpoint powers the AI-generated market analysis feature. At 26 s, users had to stare at a loading spinner long enough to lose interest. Every second saved here directly reduces

the chance of a user navigating away. The smaller prompt also reduces our per-request AWS Bedrock cost.

3. `search_tickers()` — **Added in-memory TTL cache.** We added a module-level dictionary cache with a 5-minute TTL keyed on the lowercased query string. Repeated searches for the same term now return instantly without hitting the yfinance API.

   Why this matters: Ticker search is the primary way users navigate the app — they type a company name, see results, and click through. A 220 ms delay on every keystroke (or every search submission) creates noticeable lag in what should feel like an instant autocomplete experience. With caching, repeated and common queries (e.g. "AAPL", "TSLA") resolve in sub-millisecond time.

4. `get_stock()` — **Parallelized data fetches.** The route handler previously called `fetch_bars(ticker)` and `fetch_info(ticker)` sequentially. We wrapped both in a `ThreadPoolExecutor(max_workers=2)` so they execute concurrently; wall-clock time is now the slower of the two rather than their sum.

   Why this matters: The stock detail page is the core of the application — users land here after every search. On a cache miss (first view of a new ticker, or after the 24-hour TTL expires), the user would previously wait for price history *and then* company metadata to load back-to-back. Parallelizing the two independent network calls shaves roughly 30% off the cold-load time, making the first visit to any ticker noticeably snappier.

5. `cache.get()` / `cache.set()` — **Added in-memory L1 layer.** We introduced an LRU `OrderedDict` (capacity 64 entries) as a write-through L1 cache in front of the existing file/Redis/S3 backend. On a cache hit the JSON deserialization and file I/O are skipped entirely.

   Why this matters: Nearly every API route in the application checks the cache before making an external call. The cache layer is invoked 10+ times per profiling run, and in production it is hit on every single request. Even though each individual file-I/O access was only 1–2 ms, those milliseconds compound across every route. The L1 layer eliminates this overhead for hot keys, which is especially important under concurrent load where disk I/O can become a contention point.

After: Optimized Profile

The optimized run completed in **21.00 s** (33% faster overall).

Function	Before (s)	After (s)	Speedup
`generate_market_analysis()`	25.979	16.755	1.55×
`fetch_market_summary()`	4.851	0.600	8.1×
`get_stock()`	1.392	0.934	1.5×
`search_tickers()`	0.220	<0.001	>200×
`cache.get()`	0.017	<0.001	>17×

The largest absolute gain came from parallelizing `fetch_market_summary()`, which dropped from 4.85 s to 0.60 s — an 8× improvement. `search_tickers()` and `cache.get()` both fell off the top-20 profile entirely after optimization, indicating sub-millisecond execution. The Bedrock call in `generate_market_analysis()` remains the dominant cost (16.8 s) because it is bounded by external model inference time; the 1.55× speedup comes from the reduced prompt size and token budget, which reduced both network payload and generation length.

Part Two - Showing Code Coverage

We set up automated code coverage reporting for both the frontend and backend using two separate GitHub Actions workflows.

Backend Coverage

The backend workflow (`.github/workflows/backend-coverage.yml`) uses `pytest-cov` to measure coverage of the FastAPI application. It runs on every push to `main` and on every pull request that touches the `backend/` directory. The workflow installs dependencies, then runs:

```
pytest tests --cov=app --cov-branch \
    --cov-report=term-missing \
    --cov-report=xml:coverage/coverage.xml \
    --cov-report=json:coverage/coverage.json
```

Branch coverage is enabled with `--cov-branch` so that we track not just which lines were executed but also which conditional branches were taken. The generated `coverage.json` is parsed by a GitHub Actions script step that computes line, statement, and branch percentages and formats them into a Markdown table. On pull requests this table is posted (or updated in-place) as a bot comment so reviewers can see the impact on coverage immediately. On pushes to `main` a helper Node.js script (`scripts/update-readme-backend-coverage.mjs`) rewrites the `<!-- backend-coverage:start -->` section of `README.md` with an shields.io badge and the latest numbers, then commits and pushes the change automatically.

Current backend coverage: **98.68% lines, 99.75% statements, 94.25% branches**.

Frontend Coverage

The frontend workflow (`.github/workflows/frontend-coverage.yml`) uses Jest with the `--coverage` flag (configured in `frontend/package.json` as `npm run test:coverage`). It runs on every push to `main` and on every pull request. After tests complete, a GitHub Actions script step reads `coverage-summary.json` (or falls back to `coverage-final.json`) to extract line, statement, branch, and function percentages. As with the backend, a Markdown table is posted as a bot comment on pull requests, and on merges to `main` the `<!-- coverage:start -->` section of `README.md` is updated with a fresh badge and table via `scripts/update-readme-coverage.mjs`.

Current frontend coverage: **99.08% lines, 99.08% statements, 85.55% branches, 91.33% functions**.

Demo-Only Files Excluded from Coverage

Several files and data assets in the repository exist solely to power a pre-loaded demo experience and are not part of the application's testable logic:

- `backend/app/demo_events.py` — a thin read-only helper that queries pre-seeded event rows from the database and returns them as `NewsEvent` objects. It contains no business logic; its only purpose is to serve static demo data to unauthenticated visitors so the app is immediately usable without running the full pipeline.
- `backend/demodata/` — a directory of pre-generated CSV files (e.g. `NVDA_event_news_llm_filtered.csv`) that were produced by a one-off pipeline run and committed directly. These are static seed files, not code, and have no corresponding test surface.

Because these files serve a presentation purpose rather than implementing features, writing unit tests for them would inflate coverage numbers without validating any real logic. They are acknowledged here so that the reported coverage figures (**98.86% lines** backend, **99.08% lines** frontend) are interpreted against the actual application code only.

README Badges

Both workflows write live coverage badges to the top of `README.md` using the shields.io badge URL format, so anyone visiting the repository can see the current coverage at a glance without needing to open a CI run.

Part Three: Test Coverage Depth

We chose `backend/app/pipelines/core/event_detection.py` as our most complex module. It implements the full event-detection pipeline: loading and aligning stock/market/news CSV data, normalising time arguments, detecting structural change points via PELT, building and merging index windows around those change points, scoring each window with a Cumulative Abnormal Return (CAR) model, attaching relevant news articles, and writing a ranked output CSV. The class contains six public methods plus three private helpers, each with its own branching logic and failure modes.

We wrote 20 unit tests in `backend/depth_test/test_event_detection.py` covering the following areas:

Input Validation / Time Argument Normalisation

- **test_normalize_time_arg_returns_none_for_none** — confirms that `None` is a valid optional input and passes through unchanged.
- **test_normalize_time_arg_rejects_invalid_value** — verifies that a malformed string (e.g. `"not-a-date"`) raises a `ValueError` with a descriptive message rather than silently producing `NaT`.
- **test_normalize_time_arg_removes_timezone** — ensures that a timezone-aware ISO-8601 string is stripped to a naive `Timestamp` so it can be compared with the timezone-naive price data.
- **test_init_rejects_start_time_after_end_time** — guards against reversed bounds being silently accepted; the constructor should raise immediately.

Date-Range Filtering

- **test_get_detection_df_filters_by_date_range** — with a valid `[start, end]` window, only rows whose `Date` falls inside the range should be returned.
- **test_get_detection_df_raises_for_empty_range** — a date window that matches no rows in the price dataframe must raise `ValueError("No price data available ...")`.

Window Building and Merging

- **test_build_windows_clamps_to_dataset_bounds** — change-point indices at the start/end of the series must be clamped to `[0, n-1]` so no out-of-range indices are emitted.
- **test_merge_windows_returns_empty_for_empty_input** — an empty change-point list must produce an empty merged list without crashing.
- **test_merge_windows_merges_overlapping_ranges** — overlapping tuples such as `(1,4)` and `(3,6)` must collapse into a single `[1,6]` span; non-overlapping tuples must remain separate.
- **test_build_windows_with_negative_window_values_produces_invalid_ranges** — documents a known edge case: negative `window_left`/`window_right` values invert the window bounds; the test asserts the degenerate output so the behaviour is explicit.

Change-Point Detection

- **test_detect_change_points_ignores_out_of_range_indices** — if PELT returns an index beyond the series length (e.g. `99` for a 6-row frame), it must be silently dropped; only in-bounds indices are kept.
- **test_detect_change_points_with_constant_series_returns_no_points** — a constant price series produces no breakpoints; the method should return two empty lists rather than crashing on the normalisation divide-by-zero.

CAR Scoring

- **test_score_events_skips_failures_and_sorts_by_absolute_car** — if the CAR model raises for one window, that window is skipped; surviving events are returned sorted by `abs_car` descending.
- **test_score_events_returns_empty_dataframe_when_all_car_calls_fail** — if every CAR call raises, the method must return an empty `DataFrame` with the correct columns rather than crashing.

News Attachment

- **test_attach_news_only_keeps_articles_inside_news_window** — only articles whose `published_utc` falls within `[event_date - window_days, event_date + window_days]` should be joined; articles outside must be excluded.
- **test_attach_news_with_negative_news_window_days_returns_no_matches** — a negative `news_window_days` creates an impossible date range, so the result must be empty; documents this as a known edge case.

Data Loading

- **test_load_data_raises_when_close_column_is_missing** — a stock CSV that lacks the `Close` column must propagate a `KeyError` immediately.
- **test_load_data_raises_when_news_published_utc_column_is_missing** — a news CSV without `published_utc` must raise `KeyError`.
- **test_load_data_with_no_overlapping_dates_leaves_detection_df_empty** — when stock and market CSVs cover disjoint date ranges the inner join produces no rows; a subsequent call to `_get_detection_df` then raises `ValueError`.
- **test_load_data_with_empty_csv_files_does_not_crash** — if all three CSVs raise `EmptyDataError`, `load_data` must degrade gracefully to empty dataframes with correct column schemas instead of propagating the exception.

Full Pipeline (`run`)

- **test_run_limits_to_top_k_and_writes_output** — the pipeline must truncate the event list to `top_k_events` before passing it to `attach_news`, and must write a `<TICKER>.csv` file to `results_dir`.
- **test_run_with_zero_top_k_returns_no_events_but_still_writes_output** — `top_k_events=0` must produce empty return values without crashing, and the output CSV must still be created.
- **test_run_writes_expected_csv_contents** — the CSV written to disk must contain exactly the rows returned by the pipeline, verified by re-reading and comparing column values.
- **test_run_creates_missing_results_directory_and_output_file** — if `results_dir` does not exist, `run` must create the full nested path and write the output file successfully.

Part Four: Two Memorable Bugs

link to the walk through video

Bug 1 — Empty Event-News Match Crashes the Entire Pipeline

What the bug was. In `event_detection.py`, the `attach_news` method constructs a list of result dictionaries by joining each detected event with every news article that falls within its time window. When no news articles matched any detected event — a completely legitimate outcome for low-coverage tickers or narrow date windows — `pd.DataFrame(results)` produced an empty `DataFrame` with *no columns at all* (shape `(0, 0)`). Back in `run`, the line `results_df.sort_values("event_date", inplace=True)` was called unconditionally. Because `results_df` had no columns, this immediately raised `KeyError: 'event_date'`,

crashing the entire pipeline. The `to_csv` call and the "Saved" print message were never reached, so no output file was produced for that ticker. Discovered during an end-to-end run against a thinly-covered ticker that had no recent news.

Debugging process. The pipeline terminated with an unhandled `KeyError` traceback whose innermost frame pointed to `sort_values("event_date")` inside `run`. Running the detector interactively and breaking just before `sort_values` revealed `results_df` had shape `(0, 0)` with no columns. Tracing back into `attach_news`, we confirmed the root cause: `pd.DataFrame([])` — constructed from an empty Python list — loses all column information, unlike `pd.DataFrame([], columns=[...])` which preserves schema. Because the caller (`run`) never checked whether columns existed before sorting, any ticker with zero news matches would crash the entire run.

The fix. Commit `8e6cb9a` added an early-return guard in `attach_news`:

```
if results_df.empty:
    return pd.DataFrame(
        columns=list(events_df.columns) + list(self.news_df.columns)
    )
```

And made both `sort_values` calls in `run` conditional:

```
if not events_df.empty and "event_date" in events_df.columns:
    events_df.sort_values("event_date", inplace=True)
if not results_df.empty and "event_date" in results_df.columns:
    results_df.sort_values("event_date", inplace=True)
```

What we learned. This bug taught us more than a pandas edge case: it showed that "no result" is a normal outcome in real-world data, and our system should handle it gracefully instead of treating it like an exception. We learned to design each pipeline stage with stable output contracts (even when empty), ask boundary-case questions during review (especially for empty inputs), and prioritize graceful degradation so users get reliable outputs rather than crashes.

Tests created.

- **test_attach_news_with_negative_news_window_days_returns_no_matches** — triggers the empty-return branch of `attach_news` directly and asserts the result is empty rather than raising.
- **test_run_with_zero_top_k_returns_no_events_but_still_writes_output** — exercises the `sort_values` guard in `run` by flowing an empty events frame all the way through and asserting the output CSV is still written.

Bug 2 — A Single Ticker Failure Aborts the Entire Monthly Pipeline

What the bug was: The monthly event pipeline (`run_monthly_event_pipeline.py`) loops over a list of tickers and, for each one, (1) refreshes price history, (2) ingests and cleans incremental news, and (3) runs the full event detection and LLM filtering pipeline. None of these three stages was wrapped in exception handling. If any ticker raised — for example, because the Polygon API returned a rate-limit error for one ticker, the LLM inference server timed out mid-batch, or an S3 key was unexpectedly absent — the entire loop crashed, leaving every subsequent ticker completely unprocessed. In a seven-ticker run this meant that a transient network blip against ticker #2 would silently skip tickers #3 through #7 for that entire monthly refresh cycle.

Debugging process: We discovered the issue during a full run of `run_monthly_event_pipeline`: while processing the `AAPL` ticker, one news-filtering batch triggered a bad OpenAI request and raised an unhandled exception inside the LLM filtering stage. Because that exception was not isolated at the batch or ticker boundary, it terminated the `AAPL` run and aborted the entire loop, so all remaining tickers were skipped as well. The logs made the blast radius clear: a single malformed request in one batch was able to kill the complete monthly pipeline execution.

The fix: Commit `3a5d71b` wrapped each of the three per-ticker stages in independent `try/except` blocks. A `failures` list accumulates error metadata {`ticker, stage, error`}. Any per-ticker exception is caught, logged, appended to `failures`, and the loop continues to the next ticker. A summary is printed at the end:

```
except Exception as exc:
    failures.append({"ticker": ticker,
                     "stage": "event_pipeline",
                     "error": str(exc)})
    print(f"[monthly] Event pipeline failed for {ticker}: {exc}")
```

The same resilience pattern was applied to the LLM filtering batches inside `NewsLLMFilter.filter`, so a single bad batch no longer discards results from all other batches.

What we learned

- **Enforce fault isolation:** In batch processing pipelines, independent work units (tickers, batches, files) must be strictly isolated. A failure in one unit should never expand its "blast radius" to block the processing of others.
- **Design for observability:** Accumulating structured error metadata (e.g., {`ticker, stage, error`}) instead of merely printing stack traces prevents silent failures. It paves the way for automated alerting, dashboards, and targeted retries without requiring future changes to the recovery logic.

Tests created

- **test_score_events_skips_failures_and_sorts_by_absolute_car** — verifies that a CAR failure for one event window is skipped while valid windows are still returned and sorted correctly, exercising the same continue-on-exception pattern at the window level.
- **test_score_events_returns_empty_dataframe_when_all_car_calls_fail** — confirms the worst-case: if every window fails, the pipeline degrades to an empty result rather than crashing.
- **test_main_collects_failures_and_honors_skip_flags** — verifies that the run_monthly_pipeline continues past ingestion and cleaning errors, records both failures in its summary, and skips the event-pipeline stage when that flag is set.